

DIGITAL LIBRARY SYSTEM

NestJS Endpoints Assignment

Web Development

1. Overview

In this assignment you will create two NestJS modules — Books and Users — each exposing three HTTP GET endpoints. All data must be static (no database required). The objective is to demonstrate your understanding of how HTTP endpoints are structured, routed, and tested in a NestJS application.

By the end of this exercise you should be able to:

- Generate NestJS modules, services, and controllers using the CLI
- Define GET endpoints with and/or without route parameters
- Return data from a service to a controller
- Handle cases where requested data does not exist
- Test the endpoints using Postman

2. Project Setup

Run the following commands inside your existing NestJS project to scaffold the required modules and associated files:

Generate the Books module:

```
nest g mo books
nest g s books
nest g co books
```

Generate the Users module:

```
nest g mo users
nest g s users
nest g co users
```

After running these commands your project structure should include:

```
src/
  books/
    books.module.ts
    books.service.ts
    books.controller.ts
```

```
users/  
  users.module.ts  
  users.service.ts  
  users.controller.ts
```

3. Module 1 — Books

Add the following static array as a private property inside BooksService:

```
private books = [  
  { id: 1, title: 'The Hobbit',      author: 'J.R.R. Tolkien', year: 1937},  
  { id: 2, title: '1984',    author: 'George Orwell',   year: 1949},  
  { id: 3, title: 'Harry Potter',  author: 'J.K. Rowling',    year: 1997 },  
  { id: 4, title: 'The Alchemist', author: 'Paulo Coelho',  year: 1988 },  
];
```

Endpoint 1 — Get All Books

Route:

```
GET /books
```

Expected Response:

```
[  
  { "id": 1,"title": "The Hobbit","author": "J.R.R. Tolkien","year": 1937},  
  { "id": 2,"title": "1984",  "author": "George Orwell",  "year": 1949 },  
  { "id": 3,"title": "Harry Potter","author": "J.K. Rowling","year": 1997 },  
  { "id": 4,"title": "The Alchemist","author": "Paulo Coelho","year": 1988 }  
]
```

Endpoint 2 — Get One Book by ID

Route:

```
GET /books/:id
```

Example Request: GET /books/1

Expected Response (found):

```
{ "id": 1, "title": "The Hobbit", "author": "J.R.R. Tolkien", "year": 1937 }
```

Expected Response (not found):

```
{ "message": "Book not found" }
```

Endpoint 3 — Get Books by Author

Route:

```
GET /books/author/:author
```

Example Request: GET /books/author/George Orwell

Expected Response (found):

```
[
  { "id": 2, "title": "1984", "author": "George Orwell", "year": 1949 }
]
```

Expected Response (not found):

```
{ "message": "No books found for this author" }
```

IMPORTANT: Place the `author/:author` route ABOVE the `:id` route in your controller to avoid routing conflicts.

4. Module 2 — Users

Add the following static array as a private property inside `UserService`:

```
private users = [
  { id: 1, name: 'Alice Banda', email: 'alice@library.com', role: 'admin' },
  { id: 2, name: 'Bob Phiri', email: 'bob@library.com', role: 'member' },
  { id: 3, name: 'Carol Tembo', email: 'carol@library.com', role: 'member' },
  { id: 4, name: 'David Mwale', email: 'david@library.com', role: 'admin' },
];
```

Endpoint 4 — Get All Users

Route:

```
GET /users
```

Expected Response:

```
[
  { "id": 1, "name": "Alice Banda", "email": "alice@library.com", "role": "admin" },
  { "id": 2, "name": "Bob Phiri", "email": "bob@library.com", "role": "member" },
  { "id": 3, "name": "Carol Tembo", "email": "carol@library.com", "role": "member" },
]
```

```
{ "id": 4, "name": "David Mwale", "email": "david@library.com", "role": "admin" }
]
```

Endpoint 5 — Get One User by ID

Route:

```
GET /users/:id
```

Example Request: GET /users/3

Expected Response (found):

```
{ "id": 3, "name": "Carol Tembo", "email": "carol@library.com", "role": "member" }
```

Expected Response (not found):

```
{ "message": "User not found" }
```

Endpoint 6 — Get Users by Role

Route:

```
GET /users/role/:role
```

Example Request: GET /users/role/admin

Expected Response (found):

```
[
  { "id": 1, "name": "Alice Banda", "email": "alice@library.com", "role": "admin" },
  { "id": 4, "name": "David Mwale", "email": "david@library.com", "role": "admin" }
]
```

Expected Response (not found):

```
{ "message": "No users found with this role" }
```

IMPORTANT: Place the role/:role route/method ABOVE the :id route in your controller to avoid routing conflicts.

5. Endpoints Summary

#	Method	URL	Expected Response
1	GET	/books	Array of all books
2	GET	/books/:id	Single book or not found message
3	GET	/books/author/:author	Books by author or not found message
4	GET	/users	Array of all users
5	GET	/users/:id	Single user or not found message
6	GET	/users/role/:role	Users by role or not found message

6. Testing with Postman

You must test every endpoint using Postman.

7. Further Reading

The following official documentation links will help you understand the concepts used in this assignment. You are strongly encouraged to read them before starting.

NestJS

NestJS Controllers (routing and endpoints): <https://docs.nestjs.com/controllers>

NestJS Providers and Services: <https://docs.nestjs.com/providers>

NestJS Modules: <https://docs.nestjs.com/modules>

NestJS CLI (generating files): <https://docs.nestjs.com/cli/usages>

HTTP & REST

HTTP Request Methods (GET, POST, etc.):

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>

REST API Design Basics: <https://restfulapi.net>

TypeScript

TypeScript Handbook (Classes and Decorators):
<https://www.typescriptlang.org/docs/handbook/classes.html>